

rectangle). These are the three types of cells you can use in a Jupyter Notebook. Code cells can be used for executable code. Markdown cells can be used for entering text data, and to write comments, explanations, etc. For example, if you are a computer trainer you can create interactive content of code and explanatory text, and share it with your students. Unlike in a terminal, you can edit and rerun code in Jupyter Notebook, which is especially handy when you make simple typographical errors. Figure 3 shows how a few lines of Python code are executed in a Jupyter Notebook.

The button to execute the code in a cell is marked with a blue square. In order to execute the code in a cell, select that cell and then press this button. Figure 3 shows a markdown cell marked with a red rectangle, a code cell marked with a green rectangle, and the output of the code executed marked with a yellow rectangle. In this example, the Python code just prints the value of π (Pi).

As mentioned earlier, a number of libraries and packages are available in JupyterLab by default and you don't need to install them in the beginning. You can import these libraries into your code by using the command `import`. The command `!pip freeze` will give you the complete list of libraries and packages currently available in your JupyterLab. If a library or package is not installed, then most of the time the command `pip install package/library_name_in_all_lowercase_letters` will work. For example, `pip install tensorflow` is the command to install the library TensorFlow in JupyterLab. I will explicitly mention the few times when there is a change in the installation command for a library. More powerful features of Jupyter Notebook and JupyterLab will be introduced as we proceed through this series.

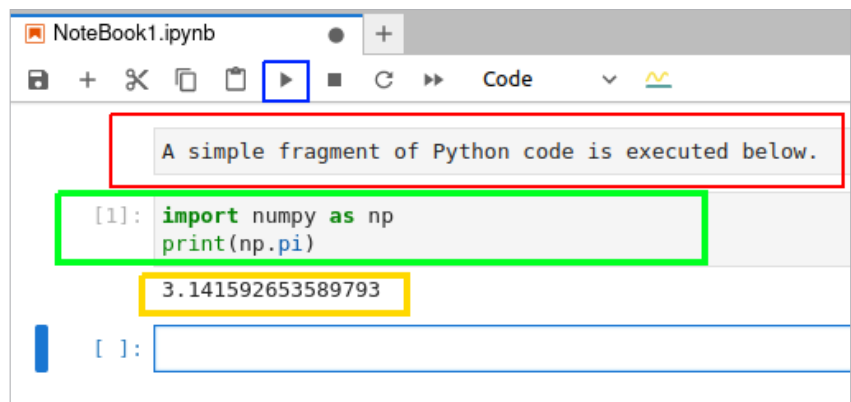


Figure 3: Python code executed in Jupyter Notebook

Some complex matrix operations

Let us learn about a few more complex matrix operations. Consider the code given below. To save some space, I am not showing the output. I have added line numbers for convenience and they are not part of the code.

```
1. import numpy as np
2. A = np.arr
   ay([[1,2,3],[4,5,6],[7,8,88]])
3. B = np.arr
   ay([[1,2,3],[4,5,6],[4,5,6]])
4. print(A.T)
5. print(A.T.T)
6. print(np.trace(A))
7. print(np.linalg.det(A))
8. C=np.linalg.inv(A)
9. print(C)
10. print(A@C)
```

First, the package NumPy is imported in line 1. Then, two matrices A and B are created in lines 2 and 3. Line 4 prints the transpose of matrix A. Compare matrix A with the transpose of A to understand how the transpose operation works. Line 5 prints the transpose of the transpose of A. You will see that A and A.T.T are equivalent. This gives you another hint about the working of the operation transpose. Line 6 prints the trace of matrix A. The value printed will be 94, because trace is the sum of the diagonal (also called main diagonal) elements of

matrix A. Notice that the main diagonal elements of A are 1, 5, and 88. Line 7 prints the determinant of A. When you execute this code, you will see that the answer is -237.00000000000009 (the value may change slightly in your machine). Since the determinant is non-zero, A is called a non-singular matrix. Line 8 stores the inverse of matrix A into matrix C. Line 9 prints matrix C. Line 10 prints the product of matrices A and C. On careful observation, you will see that the product is an identity matrix, a matrix where all the diagonal elements are 1 and all the other elements are 0. Note that exactly 1s and 0s will not be printed in the answer. For example, in the answer I got, there are numbers like -3.81639165e-17. Note that this is the scientific notation for a floating-point number, which is $-3.81639165 \times 10^{-17}$. This is -0.0000000000000000381639165, in decimal notation, which is very close to zero. Similarly, you can verify the other numbers in the answer.

On a side note, going through a tutorial on floating-point number representation in computers and the complications involved in it will definitely help you a lot and I highly recommend it. Now, by the convention adopted in the first article, we will try to classify between basic Python code and Python code for AI. In this case, all the lines of code, except lines 1 and 9, can be considered as code for AI.